Ders 1 Genişletilmiş Notu: Veri Sıkıştırmaya Giriş ve Temel Kavramlar

Ders 1 Genişletilmiş Notu: Veri Sıkıştırmaya Giriş ve Temel Kavramlar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kaynaklar ve Dersin Konuları
- 2. Veri Sıkıştırma Nedir?
- 3. Sabit Uzunluklu ve Değişken Uzunluklu Kod
- 4. Artıklık (Redundancy)
- 5. Veri Sıkıştırmada Kullanılan Kavramlar
- 6. Adaptif Olmayan, Adaptif ve Yarı Adaptif Sıkıştırma
- 7. Kayıplı ve Kayıpsız Sıkıştırma
- 8. Simetrik ve Asimetrik Sıkıştırma
- 9. Akış Modu, Blok Modu ve Birimler
- 10. Sıkıştırma Oranı, Performans ve Derlemler
- 11. Görüntü Sıkıştırmada Performans Ölçütleri
- Örnek Sorular

1. Kaynaklar ve Dersin Konuları

1.1 Kaynaklar

Slayttaki kaynak listesi üç kitaptan oluşur (ekrandan, 00:33:00):

- Khalid Sayood, *Introduction to Data Compression*
- J. Gibson, T. Berger, T. Lookabaugh, *Digital Compression for Multimedia*
- D. Salomon, *Data Compression*

Hoca ders notlarını hazırlarken en çok Salomon'un *Data Compression* kitabını kullandığını, Sayood'un kitabından da yararlandığını, ikinci kaynağı ise pek kullanmadığını söyledi. Bu kitapların farklı sürümleri bulunabilir; sürümler birbirinden farklı olsa da derste temel konular anlatılacağı için sorun olmaz. Kitaplar bazı konulara ek ayrıntılar da katar, ama dersin içinde konuların "en temel taşları" anlatılacaktır.

1.2 Dersin Konuları

Slayttaki konu listesi şöyledir (ekrandan, 00:40:00):

- Veri sıkıştırma nedir?
- Hata bulma ve düzeltme
- Temel teknikler (sezgisel yöntemler, Run-Length Coding, Move-To-Front)
- Statik kodlar (prefix, Golomb)
- İstatistiksel yöntemler (Shannon-Fano, Huffman ve türevleri, MNP ve türevleri, aritmetik kodlama, PPM)

Hoca bunların ardından sözlüksel yöntemlerin ve görüntü sıkıştırmanın geleceğini sözlü olarak ekledi.

Veri sıkıştırma nedir? bölümünde sıkıştırmanın tanımı ve sıkıştırmada kullanılan terminoloji anlatılır. Hoca bu terminolojinin önemli olduğunu, çünkü dersin geri kalanında hep bu kavramlarla konuşulacağını belirtti.

Hata bulma ve düzeltme bölümünde şu sorular ele alınacak: Bir yerden bir yere bilgi gönderilirken bozulma olabilir. Verideki bozulma tespit edilebiliyor mu, yani bozulma olduğu anlaşılabiliyor mu? Bozulduysa düzeltilebiliyor mu, düzeltilebiliyorsa kaç bit düzeltilebilir? Veri gönderilirken arkasına bir kontrol bilgisi eklenir; bunun hangi yöntemlerle yapıldığı görülecek.

Temel teknikler, sıkıştırmanın bir bilim dalı olarak ortaya çıkmasıyla birlikte önerilen başlangıç yöntemleridir. İleride görülecek yöntemlerin içinde de ara işlem adımları olarak kullanılırlar. Hoca slayttaki "Run Lenght Coding" yazımının hatalı olduğunu, doğrusunun Run-Length Encoding olduğunu ve RLE diye geçtiğini söyledi. Örneğin JPEG formatının içinde RLE de kullanılır; JPEG'de yalnız RLE yoktur, başka yöntem ve algoritmalar da vardır. Birden fazla dosyayı sıkıştırarak göndermeye yarayan RAR gibi araçlarda da bu temel tekniklerin kullanılmış olabileceğini belirtti.

Sezgisel yöntemler aslında konuya özel (ad hoc) yöntemler de olabilir: belirli bir veriye özgü geliştirilmişlerdir. Hocanın örneği şudur: bir veri yalnız cep telefonu numaralarından, başka bir veri gazete makalelerinden oluşsun. İkisini aynı yöntemle sıkıştırmak istersek başarı performansları birbirinden farklı olur. Bir girdiye özel geliştirilen yöntemin o girdide başarısı yüksek, başka girdilerde daha düşük olabilir.

Statik kodlar bölümünde prefix kodlar ve Golomb kodu görülecek (slaytta "Colomb" yazıyor). Hoca "statik kodlar varsa demek ki dinamik kodlar da olacak" diyerek kodun statik ya da dinamik olmasının ne demek olduğunun da konuşulacağını söyledi.

İstatistiksel yöntemler verinin istatistiğini kullanır. Veri bir dokümansa karakterlerin, bir görüntü dosyasıysa renklerin kullanım sıklıklarına ve olasılık değerlerine bakılır. MNP (Microcom Network Protocol) ve türevleri, aritmetik kodlama ve PPM kodlama bu grupta işlenecek. Hoca bu başlıkların altında başka yöntemler de bulunduğunu, listeye en çok yer tutanları koyduğunu söyledi.

Sözlüksel yöntemler, hocanın ifadesiyle "iki kişinin soyadından oluşmuş bir yöntem" ile anlatılacak.

> **Ek bilgi (derste anlatılmadı):** Hocanın kastettiği yöntem ailesi büyük olasılıkla Lempel ve Ziv'in soyadlarından adını alan LZ ailesidir (LZ77, LZ78, LZW gibi türevleri vardır). PPM'in açık adı kaynaklarda genellikle "Prediction by Partial Matching" olarak geçer.

Görüntü sıkıştırma aslında doküman sıkıştırmadan çok farklı değildir. En büyük fark şudur: doküman sıkıştırmada kayıp, özel durumlar dışında asla kabul edilemez; görüntü sıkıştırmada ise kayıp olabilir. Bu yüzden sıkıştırma yöntemlerini kayıplı ve kayıpsız diye de ayırmak gerekir.

1.3 Ses Sıkıştırma

Kaynak kitaplarda ses sıkıştırma da vardır, ama hoca seste çalışmadığı için dersin içeriğinde yer almaz. Sesle ilgili bir ders alanlar o dersin alt başlıklarında "voice compression" bölümlerine rastlayabilir. Lisansta sesle ilgili dersler almış ya da sesle çalışmış olanlar kendi çalışmalarını bu alana kaydırabilir. Seste de sesten çıkarılan belirli frekanslar ve özellikler üzerinde sıkıştırma yapılır; örneğin kulağın duyabildiği frekans aralığının altında ve üstünde kalan bilgiler temizlenerek sıkıştırma uygulanabilir.

2. Veri Sıkıştırma Nedir?

2.1 Tanım

Slayttaki tanım şöyledir (ekrandan, 00:50:00):

> Bir input data stream (source stream, original raw data) yerden ve zamandan kazanmak amacı ile verinin boyutunu küçülterek bir başka data stream (output, compressed) çevirme işlemidir.

Hoca tanımı şöyle açtı: Bir giriş akışından (input data stream) farklı bir akışa veri indirgenir. Arada bir işlem yapılır ve bu işleme sıkıştırma işlemi denir. Sıkıştırma iki şey sağlar: yerden tasarruf ve zamandan tasarruf.

Giriş akışı bazı kaynaklarda "source stream", bazılarında "original raw data" olarak geçer. Hoca bunların karıştırılmaması gerektiğini, üçünün de aynı şeyi, yani sıkıştırılmamış ham veriyi anlattığını vurguladı. Bu ham veri bir ses dosyası, bir doküman ya da bir görüntü dosyası olabilir. Amaca göre (yerden ya da zamandan kazanmak) seçilen bir algoritmadan geçirilerek boyutu küçültülür ve farklı bir akışa yazılır. Bu akışa output ya da compressed data, yani sıkıştırılmış veri denir.

Hoca sıkıştırmayı günlük bir benzetmeyle ayırt etti: bir çantaya kazakları koyup yumruklayarak bastırmak sıkıştırma değildir; çantanın içindeki eşyalar yine aynıdır. Veri sıkıştırmada ise verinin boyutu küçültülür ve kullanılmak istendiğinde yeniden orijinal hâline getirilir.

2.2 Algoritma Seçimi

Hoca, öğrencilerin ileride soracağı bir soruyu baştan cevapladı: "Hangi yöntemi nerede, ne zaman kullanacağımı nereden bileceğim?" Cevap uygulamaya göre değişir:

- Diskte kaplanan yer çok önemliyse seçilecek algoritma farklıdır.
- Sıkıştırılmış veri bir noktadan başka bir noktaya gönderilecekse hız önemlidir ve seçilecek algoritma yine farklıdır.
- Sıkıştırmanın uzun sürmesi önemli olmayabilir, ama açmanın çok hızlı olması istenebilir; ya da tam tersi.
- Bazı algoritmalar resim dosyasında başarılı sonuç verirken metin dosyasında aynı başarıyı göstermez.

Bu kararlar deneyimle ve algoritmaların nasıl çalıştığını kavrayarak zamanla verilebilir.

3. Sabit Uzunluklu ve Değişken Uzunluklu Kod

Sıkıştırma yapılırken, özellikle istatistiksel yöntemlerde, sembollere kod atanır. Slaytta bu konuda üç kavram verilir: fixed size code, redundancy ve variable size code (ekrandan, 00:50:00).

3.1 Sabit Uzunluklu Kod (Fixed Size Code)

Slayttaki tanım: tüm sembollerin aynı bit uzunluğunda kodlanması. Semboller örneğin A'dan Z'ye büyük ve küçük harfler, 0'dan 9'a rakamlar ya da bütün renk kodları olabilir; hepsine aynı bit uzunluğunda kod atanır.

En bilinen örnek ASCII tablosudur: her karakter 8 bittir. Küçük "a" için de büyük "A" için de 8 bit kullanılır. Türkçede daha az karşılaşılan "j" harfi için de, Türkçede ancak yabancı kelimelerde geçen "w" için de 8 bit kullanılır.

3.2 Değişken Uzunluklu Kod (Variable Size Code)

Slayttaki tanım: sembollerin farklı bit uzunluklarıyla kodlanması; kullanım sayısı fazla olan semboller daha az bitle, kullanım sayısı az olan semboller daha çok bitle kodlanır.

Hocanın örneğinde Türkçede en çok kullanılan karakter "a" olsun. "a" çok sık geldiği için 3 bitle kodlanır. "j" çok nadir geldiği için 10 bitle kodlanır. Böylece her sembolün kodlandığı bit sayısı farklı olur.

Bu bit sayılarını kim belirler? Hoca "3 bit olsun, 5 bit olsun" diye kafadan atılmadığını, bunu algoritmanın söyleyeceğini vurguladı.

4. Artıklık (Redundancy)

4.1 Tanım ve Kâğıt Benzetmesi

Slayttaki tanım: redundancy (artıklık), sıkıştırılmış veri içindeki fazlalıktır (ekrandan, 00:50:00). Hoca bunun önemli bir kavram olduğunu belirtti.

Hoca artıklığı kameraya bir kâğıt katlayarak anlattı (00:53:06 civarı). Kâğıt daha fazla küçülebiliyorsa artıklığı hâlâ fazladır. İkiye katlanınca biraz küçülür, ama hâlâ katlanabildiği için artıklığı hâlâ vardır. Bir kez daha katlanır; ince bir kâğıt olsa belki bir kez daha katlanabilirdi. Sonunda öyle bir hâle gelir ki daha fazla katlanamaz.

Veride de benzer şekilde, bazen tek bir algoritma veriyi yeterince sıkıştırır, bazen bir iki algoritmayı arka arkaya çalıştırmak gerekir. Ama hoca "sıkıştırdım sıkıştırdım, ortada bir bit kaldı" diye bir dünyanın olmadığını, tek bir bitten orijinal veriye ulaşmanın mümkün olmadığını vurguladı.

4.2 Entropi ile Artıklık Hesabı

Hoca artıklığın sayısal anlamını, henüz tanımı verilmemiş olan entropi kavramıyla anlattı (ekrandan, 00:55:40). Ekrana "$E = 3$", altına "5" ve aradaki farkı gösteren "2" yazdı.

- **Verilen:** Entropi, bu veriyi sıkıştırmak için bir sembolün ortalama 3 bitle kodlanması gerektiğini söylüyor. Tek tek semboller 1, 2, 4 ya da 5 bitle kodlanabilir, ama ortalama 3 bittir.
- **A yöntemi:** Veri A yöntemiyle sıkıştırıldığında semboller ortalama 5 bit çıkıyor. Artıklık $5 - 3 = 2$. Hoca bunun oldukça büyük bir sayı olduğunu söyledi.
- **B yöntemi:** B yöntemiyle ortalama bit uzunluğu 3,5 çıkıyor. Artıklık $3{,}5 - 3 = 0{,}5$.
- **Sonuç:** Artıklığın en küçük olduğu yöntem, yani B seçilir; bu daha iyi bir sıkıştırma performansı demektir.

Başka bir yol da A'nın çıktısının sonuna bir C algoritması eklemektir; bu da artıklığı düşürebilir. Ama tek başına 0,5 artıklık veren bir yöntem varken iki yöntemi arka arkaya kullanmanın anlamı olmayabilir, çünkü amaç zamandan da tasarruf etmektir. Genelde bir algoritmanın çalışma süresi iki algoritmanınkinden azdır, ama tersi de olabilir: bazı algoritmalar o kadar karmaşıktır ki iki algoritmayı çalıştırmaktan daha uzun sürer. Bu yüzden hoca önce ihtiyacın belirlenmesi gerektiğini tekrarladı: öncelik yerden mi, zamandan mı, yoksa ikisinden birlikte mi kazanmaktır?

4.3 Sembollerin Türü Hakkında Soru

Soru: Harfler, özel karakterler (yıldız gibi) ve rakamlar için sıkıştırmada farklı yöntemler ya da farklı bit uzunlukları kullanılıyor mu?

Cevap: Hoca "a" neyse yıldızın da o olduğunu, boşluk neyse eğik çizginin de o olduğunu söyledi; hepsine karakter gözüyle bakılır. İstatistiksel bir yöntemde önemli olan sembolün sıklığıdır: "a", eğik çizgiden daha sık geçiyorsa "a"yı daha az bitle kodlamak daha önemlidir. Görüntü sıkıştırmada da benzer bir durum vardır: bir resimde kırmızının 10 farklı tonu olabilir, ama insan gözü hepsini ayırt edemiyorsa bu 10 ton sanki 3 farklı kırmızı varmış gibi en yakın baskın kırmızıyla kodlanabilir ve göz farkı algılamaz. Siyah bir boyaya çok az gri katıldığında renk yine siyah görünür. Sonuçta her renk de bir sembol olarak görülür.

5. Veri Sıkıştırmada Kullanılan Kavramlar

Slayttaki kavramlar şunlardır (ekrandan, 01:00:00):

- **Compressor / Encoder / Kodlayıcı:** Input stream ham veriyi sıkıştıran ve sıkıştırılmış veri ile output stream oluşturan algoritma. Hoca, kaynak kitapların bazılarının "encoder", bazılarının "compressor" terimini kullandığını belirtti.
- **Decompressor / Decoder / Kod çözücü:** Kodlayıcının yaptığı işi ters yönde yapar. Output stream'deki veriyi alır, ters yönde işlem yaparak orijinal hâle dönüştürür.
- **Codec:** Encoder ve decoder'ı birlikte tanımlar.
- **Unencoded / Raw Data / Original Data:** Orijinal giriş verisi.

- **Encoded / Compressed:** Sıkıştırılmış veri.

Hoca “ters işi yapar” ifadesinin her zaman birebir aynı işlemlerin tersini yapmak anlamına gelmediğini, bazen açarken farklı işlemler yapılması gerektiğini söyledi. Bu konu simetrik ve asimetrik sıkıştırmada tekrar ele alındı.

Codec için şu tabloyu çizdi: blok şemalarında bazen içinde “codec” yazan bir kutu görülür. Veri kutuya girer, içeride sıkıştırılır ve çözülür, dışarıya sonuç verilir. Kutu hem kompresörü hem dekompresörü birlikte gerçekleştirir; içeride yapılan işlemlerin ayrıntısını bilmek gerekmez, tek bir kapalı kutu olarak gösterilir.

6. Adaptif Olmayan, Adaptif ve Yarı Adaptif Sıkıştırma

6.1 Slayttaki Tanımlar

Slayttaki tanımlar şunlardır (ekrandan, 01:20:00):

- **Non-adaptive compression method:** Parametre ve işlemler belli tip veriler için geçerlidir (facsimile compression).
- **Adaptive compression:** İşlenmemiş her türlü veri üzerinde işlem yapar. Parametre ve işlemler değiştirilebilir.
- **Semi-adaptive compression:** İki adımda işlemler gerçekleştirilir. Önce veri üzerinden geçilerek tüm semboller için kullanım sıklıkları çıkarılır, sonrasında sıkıştırma işlemi gerçekleştirilir.

6.2 Adaptif Olmayan Sıkıştırma ve Faks

Adaptif olmayan bir algoritma çalışırken bazı parametrelerin baştan belirlenmesi gerekir; belirli tipte işlemler yapılır ve belirli tipteki veri üzerine uygulanır. Slayttaki örnek faks (facsimile) sıkıştırmasıdır.

Faks çekilirken bir A4 sayfa makineye konur. Faks makinesinin içinde bir sıkıştırma algoritması çalışır: sayfayı alır, sıkıştırır ve karşı tarafa gönderir. Karşı taraftaki faks sıkıştırılmış veriyi alır ve orijinal hâline açar. Hoca şunu vurguladı: faks, elindeki veri ister görüntü ister yazılı metin olsun, onu her zaman görüntü olarak algılar. Görüntü siyah-beyaz, gri seviyeli ya da renkli olabilir; yazılı bir metin doküman olarak değil, bir resim dosyası olarak kabul edilir.

Resimdeki satırlara scan line denir. Bir karakterin yazıldığı alanın yüksekliği birkaç scan line’a karşılık gelir; hoca bunun kullanılan parametreye göre 7, 8 ya da 10 scan line olabileceğini, kesin sayıyı hatırlamadığını söyledi. O alandaki piksel değerleri kullanılarak karakterin ne olduğu zaten çözümlenebilir.

6.3 Adaptif ve Yarı Adaptif Sıkıştırma

Adaptif bir yöntemde algoritma veriye göre kendine bir yol çizer. Veri bir görüntüyse onun görüntü olduğunu algılayarak (ya da kullanıcı bunu belirterek) arka planda buna uygun bir algoritma çalışır; doküman ise doküman olduğunu bilerek çalışır. Parametreler ve işlemler algoritmayı seçen kişiler tarafından belirlenir.

Yarı adaptif (semi-adaptive) yöntem ne adaptif ne de adaptif olmayandır; işlem iki adımda gerçekleşir. Birinci adımda veri üzerindeki tüm sembollerin kullanım sıklıkları çıkarılır ve elde istatistiksel bir bilgi oluşur. İkinci adımda bu bilgi kullanılarak semboller kodlanır ve sıkıştırma başlar. Örneğin dokümanda “n” harfinin kaç kez geçtiği tek tek sayılır, sonra bütün “o”lar sayılır; her karakterin kullanım olasılığı çıkarılır. Bu birinci aşamadır. İkinci aşamada yöntem bu olasılıklar üzerinde çalıştırılır.

Adaptif yöntemlerde de istatistik bilgisi çıkarılır, ama farklı biçimde. Akış okunurken o ana kadar “n” bir kez geçtiyse ona bir kod verilir. Üçüncü karakterde “n” sayısı ikiye çıkınca “n”ye farklı bir kod verilir; daha ilerde sayısı üç olunca kodu yine değişir. Yani adaptif yöntemde “n” doküman içinde her zaman aynı kodla kodlanmaz; dokümanın farklı yerlerinde farklı kodlarla, farklı uzunluklarla karşımıza çıkabilir.

Adaptif olmayan yöntem ise dokümanın içine hiç bakmaz; büyük bir derlem (corpus) üzerinden önceden çıkardığı kararlarla işlem yapar.

6.4 “ababc” Örneği

Hoca üç türü küçük bir örnekle anlattı. Önce slayta “abab” yazdı (ekrandan, 01:21:40), sonra sona bir “c” ekleyerek “ababc” yaptı ve yanına sembol sayılarını yazdı (ekrandan, 01:23:40).

Adaptif olmayan yöntem: Elimizde bir Türkçe doküman topluluğu olsun; içinde 10 milyon A4 büyüklüğünde dosya bulunsun. Yöntem bütün bu dosyalarda “a” ve “b”nin kaç kez geçtiğine bakarak olasılıklarını bulur ve “a”ya ve “b”ye bir kod atar. Bundan sonra karşısına ne gelirse gelsin “a”yı hep aynı kodla, “b”yi hep aynı kodla kodlar. ASCII tablosu da böyledir: ekranda gördüğümüz metindeki her karakter, önceden hazırlanmış aynı bit uzunluğundaki ASCII kodlarıyla yazılmıştır.

Yarı adaptif yöntem: Önce dokümana bakar ve sayar:

- “a”: 2 kez, olasılık $2/5 = 0{,}4$
- “b”: 2 kez, olasılık $2/5 = 0{,}4$
- “c”: 1 kez, olasılık $1/5 = 0{,}2$

Bu olasılıklara göre örneğin birine 3 bitlik, diğerine 5 bitlik bir kod atar ve dokümanı sıkıştırır.

Adaptif yöntem: Karakterleri sırayla okur.

- İlk “a” gelir. O ana kadar gelen tek karakter “a” olduğu için olasılığı %100’dür; hoca bunu 1 bitle kodladığını söyledi.
- Sonra “b” gelir; belki 2 bitle kodlanır.
- Sonra yine “a” gelir. Üç karakterden ikisi “a” olduğu için olasılığı $2/3$ olur; “a”nın kodu değişir. Hoca burada “a”nın artık farklı uzunlukta, örneğin 5 bitle kodlanabileceğini söyledi.
- Bir “b” daha gelir; “b”nin sayısı 1’den 2’ye çıkar, olasılığı değişir ve ona farklı bit uzunluğunda bir kod verilir.

Dokümanın sonuna kadar her sembolün kodu defalarca değişebilir.

> **Ders sonrası not:** Hocanın bu örnekte söylediği bit sayıları (1, 2, 5 bit) açıklama amaçlıdır. Olasılığın $1 \to 1/2 \to 2/3$ biçiminde değiştiği hesaplanabilir (ara adım eklendi: “ab” okunduğunda $P(a) = 1/2$); gerçek kod uzunluklarını büyük olasılıkla kullanılan algoritma (örneğin adaptif Huffman) belirler. Olasılığı artan bir sembolün kodu genelde kısalır, uzamaz.

6.5 Karşılaştırma

Hoca üç türü şöyle karşılaştırdı:

- En iyi sıkıştırmayı adaptif algoritma yapar; kodlaması en zor olan da odur.
- Kodlaması en kolay olan adaptif olmayan algoritmadır; sıkıştırma performansı en düşük olan da odur.
- Aradaki fark büyüktür: hoca örnek olarak adaptif bir yöntemin %70’ler civarında, adaptif olmayan bir yöntemin %40 civarında sıkıştırma verebileceğini söyledi.

Soru: Sıkıştırmada büyük harf küçük harf duyarlılığı var mı?

Cevap: Hoca “elbette var” dedi; büyük ve küçük harf ayrı semboller olarak ele alınır.

7. Kayıplı ve Kayıpsız Sıkıştırma

7.1 Tanımlar

Slayttaki tanımlar (ekrandan, 01:20:00):

- **Lossy (kayıplı):** Sıkıştırılmış veri açıldığında orijinal veriye dönülemez. Ses ve görüntü sıkıştırmada kullanılır.
- **Lossless (kayıpsız):** Sıkıştırılmış veri açıldığında orijinal hâlindedir. Doküman sıkıştırmada kullanılır.

Hoca ekrana şu çizimi yaptı (ekrandan, 01:28:40): $P \rightarrow [C] \rightarrow P_s \rightarrow [DC] \rightarrow P$ ya da P'. Düz metin P bir kompresörden geçirilir, sıkıştırılmış veri P_s elde edilir. P_s bir dekompresörden geçirilir. Sonuçta P'nin birebir aynısı elde edilirse bu kayıpsız sıkıştırmadır. Orijinalinden farklı bir P' elde edilirse kayıplı sıkıştırmadır.

7.2 Hangi Veride Hangisi?

Görüntü verilerinde kayıplı sıkıştırma kullanmak genelde idealdir, ama tıbbi görüntülerde doğru olmaz. Hocanın örneğinde bir beyin MR görüntüsünde mikron seviyesinde küçücük bir kitle vardır. Sıkıştırma bunu gürültü sayıp atarsa hasta bir insanın verisi sağlıklı bir insanın verisi gibi görünür ve doktor bakıp "bir şeyiniz yok" der. Buna karşılık duvarda asılı true color bir resimde gözün ayırt edemediği birçok renk vardır; bunları aynı renk gibi görürüz ve orada kayıplı sıkıştırma uygulanabilir.

Dokümanda kayıplı sıkıştırma yapılırsa açıldığında okunur bir şey görülmeyebilir. Bazı kelimeler gerçek kelimelere benzetilerek okunabilir, ama birçok kelimenin anlamı değişir: "i" yerine "o", "n" yerine "m" gelebilir. Bu yüzden dokümanlar kayıpsız sıkıştırılır.

Dokümanda kayıplı sıkıştırma ancak içeriği bozmayacak bir durum varsa olabilir. Örneğin metindeki fazla boşluklar sıkıştırmadan önce kaldırılıp açarken tek boşluk olarak gösterilirse içerikten hiçbir şey kaybedilmez. Ama metinde "bir milyon" yazıyorsa ve kayıplı sıkıştırma sırasında iki üç sıfır kaybolursa, bir milyon TL'lik para bin TL olarak görünür; bu kabul edilemez. Hoca bu yüzden ne zaman kayıplı, ne zaman kayıpsız algoritma seçileceğinin çok önemli olduğunu vurguladı.

> **Hocanın vurgusu:** Doküman sıkıştırmada kayıp, özel durumlar dışında asla kabul edilemez; görüntü ve seste kayıp olabilir. Hangi durumda hangisinin seçileceği çok önemlidir.

8. Simetrik ve Asimetrik Sıkıştırma

8.1 Tanımlar

Slayttaki tanımlar (ekrandan, 01:20:00):

- **Symmetrical compression:** Kompresör ve dekompresör aynı algoritmayı ters yönde kullanırlar.
- **Asymmetric compression:** Kompresör ve dekompresör farklı algoritma kullanır.

Simetrik çalışmayı hoca şu çizimle gösterdi (ekrandan, 01:28:40): $a \rightarrow b \rightarrow P_s \rightarrow b \rightarrow a$. Sıkıştırırken önce A işlemi, arkasından B işlemi yapılır ve P_s elde edilir. P_s'den geri dönmek için önce B'nin tersi, sonra A'nın tersi uygulanır. Burada bir aynalama vardır: aynı adımlar birebir ters sırada yapılarak orijinal veriye dönülür.

Asimetrik yöntemde sıkıştırırken ve açarken yapılan işlemler birbirinin birebir tersi olmayabilir; bazen kayıplar da olabilir. Hocanın örneği: bir resim dosyası birtakım filtrelerden geçirilir, ortaya float sayılar çıkar, bunlar integer'a çevrilerek kullanılır. Açarken integer sayılarla işlem yapıldığı için orijinal veri zaten elde edilemez. Bu durum hem kayıplı hem de asimetrik bir sıkıştırmadır. Ama asimetrik bir yöntem kayıpsız da olabilir: veri açıldığında orijinal hâline döner, fakat açma adımları sıkıştırma adımlarının ters sırası değil, farklı işlem adımlarıdır.

8.2 Uygulama Alanları

Hoca slayttaki üç soruyu derste tartıştı; sorular ve cevaplar "Örnek Sorular" bölümündedir. Kısaca:

- Kodlayıcı karışık ve uzun, kod çözücü basit ve hızlıysa: arşiv uygulamaları.
- Kodlayıcı basit ve kısa, kod çözücü karışık ve yavaşsa: yedekleme (backup) uygulamaları.
- Sıkıştırma performansı önemli, hız önemsizse: hoca eskiden yazılımların disket ve CD'lerle dağıtıldığı dönemi örnek verdi.

Hoca bugünün güçlü donanımlarında aradaki bekleme sürelerinin saatler değil, örneğin 10 saniyeye karşı 20 saniye gibi farklar olduğunu, çok yıllar önce durumun farklı olduğunu ekledi.

Soru: Sıkıştırma ve açma süreleri farklıysa, kesin olarak asimetrik bir algoritma kullanılmış diyebilir miyiz?

Cevap: Hoca, simetrik algoritmada aynı işlemlerin birebir ters yönde yapıldığını hatırlatarak süreler farklıysa asimetrik denebileceği görüşüne "bence evet" diye katıldı.

9. Akış Modu, Blok Modu ve Birimler

9.1 Stream Mode ve Block Mode

Slayttaki tanımlar (ekrandan, 01:40:00):

- **Stream mode:** Veri byte byte çekilir ve üzerinde işlem yapılır.
- **Block mode:** Veri bloklar hâlinde çekilir ve her blok üzerinde farklı algoritma çalıştırılabilir.

"Stream" İngilizcede akış demektir. Stream modda giriş verisinden veri byte byte çekilir: bir karakter okunur, üzerinde işlem yapılır, sonra ikinci karakter okunur ve işlenir. Süreci hızlandırmak için önbellek (cache) kullanılabilir: veri hızla önbelleğe çekilir, oradan byte byte alınır.

9.2 Bits per Character ve Bits per Pixel

Slayttaki tanımlar (ekrandan, 01:40:00):

- **Bits per chars:** Sıkıştırılan her bir sembolün ortalama bit uzunluğu.
- **Bits per pixels:** Sıkıştırılan her bir pikselin ortalama bit uzunluğu.

Hoca bunları birim olarak tanıttı: boyun santimetreyle, uzaklığın kilometreyle ölçülmesi gibi, veri sıkıştırmanın da birimleri vardır. Dokümanlar üzerinde çalışılıyorsa birim bpc'dir: bir karakter için ortalama gereken bit sayısı. "Bu algoritma 3 bits per character" demek, bir karakteri kodlamak için ortalama 3 bit gerektiği demektir.

Resimlerde karakter yerine piksel alınır ve birim bpp olur: bir piksel için kaç bite ihtiyaç var? Hocanın örnekleri:

- Siyah-beyaz bir resimde bir piksel 1 bitle kodlanır: 0 siyah, 1 beyaz.
- 8 farklı renk kullanılıyorsa $2^3 = 8$ olduğundan piksel 3 bittir.
- True color bir resim 24 bittir. Sıkıştırma sonucunda bir piksel ortalama 12 bitle gösterilirse 24 bitlik piksel ortalama 12 bitle kodlanmış olur.

10. Sıkıştırma Oranı, Performans ve Derlemler

10.1 Sıkıştırma Oranı ve Sıkıştırma Performansı

Slayttaki tanımlar (ekrandan, 01:40:00):

$$\text{compression_ratio} = \frac{\text{size of the output stream}}{\text{size of the input stream}}$$

$$\text{sıkıştırma performansı (yüzdesel)} = 100 \cdot (1 - \text{compression_ratio})$$

Hocanın örneği (ekrandan, 01:44:20):

- **Verilen:** 100 birimlik bir veri sıkıştırılmış ve 40 birime inmiştir.
- **Sıkıştırma oranı:** $\frac{40}{100} = 0{,}4$.
- **Performans:** $1 - 0{,}4 = 0{,}6$; $0{,}6 \cdot 100 = \%60$.
- **Sonuç:** Verinin %60'lık kısmı yok olmuştur; %60 yer kazanılmıştır.

Hoca oranın yüzdesel bir değer olmadığını vurguladı: oran, 100 birimlik bir şeyin 40 birime düştüğünü gösterir, sıkıştırma yüzdesini değil. Ne kadar kazanç elde edildiğini, yani ne kadar yer kazanıldığını görmek için oranın 1'den farkı alınmalıdır.

> **Hocanın vurgusu:** Compression ratio yüzdesel olarak ne kadar sıkıştırıldığını göstermez; kazancı bulmak için mutlaka oranın 1'den farkı alınmalıdır.

10.2 Derlemler (Corpus)

%60 başarı elde eden biri bununla sevinmeli mi, üzülmeli mi? Hoca, "en iyi yöntemi buldum" diyen birine kimsenin hemen inanmayacağını, inandırabilmek için hazırlanmış derlemler bulunduğunu söyledi. Slayttaki derlemler: Calgary, Canterbury, UCI (ekrandan, 01:46:00).

Hoca Calgary ve Canterbury'nin Kanada'daki üniversitelerin geliştirdiği derlemler olduğunu, Canterbury'nin daha sonra çıktığını söyledi. Canterbury'de yalnız düz metin değil, Java kodu ve XML biçiminde dosyalar gibi farklı formatlarda dosyalar da vardır. UCI'yi ise Kaliforniya'daki bir üniversitenin ortaya attığını ve genelde görüntü işlemeyle ilgili olduğunu söyledi.

Bu standart veri setlerinin mantığı şudur: yeni bir algoritma geliştiren herkes onu bu veri setleri üzerinde çalıştırır ve elde ettiği başarıyı başkalarının başarısıyla kıyaslar. Böylece "şu tip algoritmayı kullanan birinden daha iyi, şundan daha kötü sonuç aldım" ya da "onun algoritması şu tür veride daha başarılı, benimki şu tür veride" denebilir. Ortak bir taban (base) olmazsa herkes kendi yöntemine en iyi sonucu verecek veri setini kendisi oluşturabilir.

> **Ders sonrası not:** Canterbury derlemi büyük olasılıkla Yeni Zelanda'daki University of Canterbury'de hazırlanmıştır; Calgary derlemi ise büyük olasılıkla Kanada'daki University of Calgary'de. UCI kısaltması büyük olasılıkla University of California, Irvine'ı gösterir; bu kurumun bilinen veri deposu yalnız görüntü verisiyle sınırlı değildir.

11. Görüntü Sıkıştırmada Performans Ölçütleri

Hoca, şimdiye kadar verilen sıkıştırma oranı ve performansının genelde metinlerdeki başarıyı ölçmek için kullanıldığını, resimlerde başka ölçütlere bakıldığını söyledi.

11.1 Hata ve Total Error

Ekrandaki tanımlar şunlardır (ekrandan, 01:50:00): $I(r, c)$ orijinal resim, $I'(r, c)$ decompress edilmiş resimdir. Resim satır (row) ve sütunlardan (column) oluşur.

Resim kayıplı bir yöntemle sıkıştırılıp açıldığında orijinali elde edilemez. Ama resmin boyutu değişmez; değişen piksellerin değeridir. Örneğin orijinalde 100 farklı renk varken açılan resimde belki 60 farklı renk vardır; resim yine aynı boyuttadır ama daha az sayıda renkle oluşturulmuştur. İki resmin karşılıklı pikselleri arasındaki farka hata (error) denir:

$$\text{error}(r, c) = I'(r, c) - I(r, c)$$

$M \times N$ boyutlu resim için sıkıştırıldıktan sonraki toplam hata ekranda şöyle verilir:

$$\text{total error} = \sum_{r=0}^{M-1} \sum_{c=0}^{N-1} [I'(r, c) - I(r, c)]$$

11.2 Neden Kare Alınır?

Hoca ekrana $5-4=1$ ve $4-5=-1$ yazdı (ekrandan, 01:50:00). İki durumda da aradaki fark 1'dir, ama aritmetik olarak biri artı, biri eksidir. Farklar doğrudan toplanırsa $1+(-1)=0$ olur ve hata yokmuş gibi görünür (ekrandan, 01:55:20). Hoca bunu sayısal analizdeki regresyonda kullanılan en küçük kareler yöntemine, yani farkların karelerinin toplamına benzetti. Karesi alınınca $1^2 = 1$ ve $(-1)^2 = 1$ olur; sonuç değişmez. Ekrandaki ikinci örnekte farklar 2 ve -2'dir; kareleri 4 ve 4'tür.

Soru (hocanın kendi sorduğu): Neden mutlak değer değil de kare alınıyor?

Cevap: Hoca yöntemin karelerle gittiğini, karelerin alınmasının büyük olasılıkla gerçek değere daha iyi yaklaştığı için tercih edilmiş olabileceğini söyledi; kesin bir gerekçe vermedi.

11.3 Root Mean Square Error

$$e_{RMS} = \sqrt{\frac{1}{N^2} \sum_{r=0}^{M-1} \sum_{c=0}^{N-1} \left[I'(r,c) - I(r,c)\right]^2}$$

Hoca formüldeki N^2'nin aslında $N \cdot M$ olduğunu söyledi. Formülü basitleştirmek için resmin kare ($N \times N$) olduğu varsayılır; $N \times M$ boyutlu bir resim padding ile kare hâle getirilebilir. Bu yüzden N^2 yazılması yanıltmamalıdır. Farkların karelerinin toplamı resim boyutuna bölünür ve karekökü alınır.

11.4 Signal to Noise Ratio ve PSNR

e_{RMS}'nin tersi bir ölçüt de SNR'dir (signal noise ratio). Pay kısmında açılmış resimdeki piksellerin karelerinin toplamı, payda kısmında orijinal ile açılmış resmin karşılıklı piksellerinin farklarının karelerinin toplamı vardır. Bunlar oranlanır ve sayısal bir değer çıkar:

$$SNR_{RMS} = \sqrt{\frac{\sum_{r=0}^{M-1} \sum_{c=0}^{N-1} I'(r,c)^2}{\sum_{r=0}^{M-1} \sum_{c=0}^{N-1} \left[I'(r,c) - I(r,c)\right]^2}}$$

Görüntüyle çalışanların bir kısmı SNR yerine PSNR (peak SNR) kullanır. Farkı, pay kısmındaki ifadenin $(L-1)^2$ olmasıdır. L resimdeki farklı renk sayısıdır; renkler sıfırıncı renkten başladığı için 1 çıkarılır. Paydada yine karşılıklı piksellerin farklarının karelerinin ortalaması vardır. Sonucun onluk tabanda logaritması alınıp 10 ile çarpılır:

$$PSNR = 10 \log_{10} \frac{(L-1)^2}{\frac{1}{N^2} \sum_{r=0}^{M-1} \sum_{c=0}^{N-1} \left[I'(r,c) - I(r,c)\right]^2}$$

Hoca SNR ya da PSNR'den birinin kullanılabileceğini, ikisinin aynı şekilde yorumlandığını, PSNR'nin logaritmik ölçekte sonuçları daha rahat gösterdiği için tercih edildiğini söyledi.

> **Ders sonrası not:** Ekrandaki SNR_{RMS} formülünün paydasında farkların karesi yazılmamıştı; hoca anlatımda farkların karelerinin alındığını söyledi, burada kareli biçim yazıldı. Ekrandaki SNR ve PSNR formüllerinde iç toplamın üst sınırı "$R-1$" olarak geçiyordu; burada $N-1$ olarak yazıldı.
>
> **Ek bilgi (derste anlatılmadı):** PSNR formülündeki $L-1$, bir pikselin alabileceği en büyük değerdir; örneğin 8 bitlik gri seviyeli bir görüntüde $L = 256$ ve $L-1 = 255$'tir. PSNR genellikle desibel (dB) cinsinden verilir.

11.5 Yorum

Hoca e_{RMS} ile SNR'nin ters orantılı olduğunu söyledi: biri küçülürken diğeri büyür (ekrandan, 01:55:20: "$e_{RMS} \downarrow$, SNR $\uparrow$"). e_{RMS} ne kadar küçük ya da SNR ne kadar yüksekse o kadar iyi bir sıkıştırma yapılmış demektir. Yayınlarda kimi sonuçlarını e_{RMS} üzerinden, kimi SNR üzerinden verir; ikisi bilinen ölçütler olduğu için ayrıca açıklanmaları gerekmez. SNR değerinin büyüdüğünü görüp "bu çok kötü" dememek gerekir; SNR büyüdükçe e_{RMS} küçülür.

> **Ders sonrası not:** Hoca e_{RMS} küçükse "redundansı az demektir, buna daha fazla sıkıştırma uygulayamazsın" dedi. e_{RMS}, SNR ve PSNR büyük olasılıkla artıklığı değil, açılan görüntünün orijinale ne kadar yakın olduğunu, yani yeniden oluşturma kalitesini ölçer.

11.6 Sonuçları Raporlarken Dikkat

Hoca karşılaştığı bir yayında, metinde sıkıştırma performansı denilip tablolarda yüzdelerin verildiğini, sonuçların anlatılan yönteme göre beklenenden çok düşük göründüğünü anlattı. Açıklama istendiğinde yazarın değeri farklı bir tanımla hesapladığı ortaya çıkmış; bu da sonucun yanlış yorumlanmasına yol açmış. Bu yüzden kullanılan ölçüt, gerekiyorsa formülüyle birlikte açıkça verilmelidir. Bu ölçütler görüntü işlemede de kullanılır.

Örnek Sorular

Soru 1 (ekrandan, 01:30:00): Kodlayıcının karışık ve uzun sürdüğü, buna karşılık kod çözücünün basit ve hızlı çalıştığı sıkıştırma algoritmalarının tercih edildiği bir uygulama alanı ne olabilir?

Derste bir öğrenci önizleme (preview) gibi bir uygulamayı önerdi. Hoca büyük kurumlarda daha önce birilerinin attığı bilgilere ihtiyaç duyulduğunu hatırlatarak cevabı verdi ve slayta yazdı:

> **Cevap (derste verildi, ekrandan, 01:32:25):** Arşiv uygulamaları. Veri arşive bir kez atılır; bu işlem bir kez yapıldığı için algoritmanın uzun sürmesi normaldir. Ama arşive atılan bilgi gün içinde farklı kişiler tarafından sürekli indiriliyorsa açma işleminin çok hızlı olması gerekir.

Soru 2 (ekrandan, 01:32:25): Kodlayıcının basit ve kısa sürdüğü, buna karşılık kod çözücünün karışık ve yavaş çalıştığı sıkıştırma algoritmalarının tercih edildiği bir uygulama alanı ne olabilir?

> **Cevap (derste verildi, ekrandan, 01:34:40):** Backup (yedekleme) uygulamaları. Bankalar her akşam gün sonu yedeğini alır; her akşam aynı işlem yapıldığı için yedek almanın kısa sürmesi gerekir. Eski bilgiye dönmek ise belki yılda bir kez ya da hiç gerekmez; yalnız tehlikeli bir anda veriyi kurtarmak için önemlidir. Her gün tekrarlanmadığı için açmanın uzun sürmesi kabul edilebilir.

Soru 3 (ekrandan, 01:34:40): İyi bir sıkıştırma performansına sahip bir algoritmanın önemli olduğu, fakat kodlama ve kod çözme hızının önemli olmadığı bir duruma örnek veriniz.

Derste bir öğrenci tomografi gibi tıbbi verilerin saklanmasını önerdi; hoca arşivde az yer kaplamasının istendiği bu tür örneklerin de olabileceğini söyledi.

> **Cevap (derste verildi, 01:35:55):** Hocanın örneği eski yazılım dağıtımıdır. Eskiden yazılımlar indirilmez, disketlerle ve CD'lerle gelirdi; CD'lerin kapasitesi de bugünkü gibi değildi. Bir yazılım yüklenirken elde üç CD olabilirdi; çok iyi bir sıkıştırma algoritmasıyla sıkıştırılsa tek bir CD'ye sığardı. Burada önemli olan az yer kaplamaktır; sıkıştırma ve açma süresi o kadar önemli değildir. Hoca günümüzde donanım geliştiği için artık bununla uğraşılmadığını ekledi.

Soru 4 (derste sözlü, 00:55:10): Entropiye göre bir sembol ortalama 3 bitle kodlanmalıdır. A yöntemi ortalama 5 bit, B yöntemi ortalama 3,5 bit veriyor. Her yöntemin artıklığı nedir ve hangisi seçilmelidir?

> **Cevap (derste verildi):** A için artıklık $5 - 3 = 2$, B için $3{,}5 - 3 = 0{,}5$'tir. Artıklığı en küçük olan B seçilir.

Soru 5 (derste sözlü, 01:43:25): 100 birimlik bir veri sıkıştırılıp 40 birime iniyor. Sıkıştırma oranı ve yüzdesel sıkıştırma performansı nedir?

> **Cevap (derste verildi, ekrandan, 01:44:20):** Oran $40/100 = 0{,}4$; performans $100 \cdot (1 - 0{,}4) = \%60$.

Soru 6 (derste sözlü, 01:37:33): Sıkıştırma ve açma süreleri birbirinden farklıysa, asimetrik bir algoritma kullanıldığı kesin olarak söylenebilir mi?

> **Cevap (derste verildi):** Hoca "bence evet" dedi: simetrik algoritmada aynı işlemler birebir ters yönde yapılır.

Dersin Toparlaması

Bu ilk derste veri sıkıştırmanın tanımı ve dersin geri kalanında kullanılacak terminoloji kuruldu. Sıkıştırma, ham bir giriş akışını yerden ve zamandan kazanmak için daha küçük bir çıkış akışına çevirmektir ve sıkıştırılan veri yeniden açılabilmelidir. Sabit uzunluklu kod (ASCII gibi) sembollerin sıklığını kullanmazken, değişken uzunluklu kod sık sembollere kısa, seyrek sembollere uzun kod verir; kod uzunluklarını algoritma belirler. Artıklık, bir algoritmanın ortalama bit uzunluğu ile entropinin söylediği değer arasındaki farktır ve en küçük artıklığı veren yöntem seçilir.

Ardından sıkıştırma yöntemlerinin sınıflandırılmasında kullanılan eksenler görüldü: adaptif olmayan, yarı adaptif ve adaptif yöntemler; kayıplı ve kayıpsız sıkıştırma; simetrik ve asimetrik sıkıştırma; akış ve blok modu. Ölçüm tarafında metin için bpc, görüntü için bpp birimleri, sıkıştırma oranı ve yüzdesel performans, karşılaştırma için standart derlemler (Calgary, Canterbury, UCI) ve görüntüde e_{RMS}, SNR ve PSNR ölçütleri anlatıldı. Hoca artıklığı anlatırken entropinin tanımının henüz verilmediğini söyledi; entropi ilerleyen derslerde tanımlanacak. Konu listesine göre sıradaki başlıklar hata bulma ve düzeltme ile temel tekniklerdir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.